

Implementación y análisis comparativo de rendimiento de primitivas RSA y ECC

Trabajo Fin de Grado - Ingeniería Informática

Leon Elliott Fuller

Junio de 2026

Tutor: Jesús García Miranda

E.T.S. de Ingenierías Informática y de Telecomunicación

Universidad de Granada

1. Motivación y objetivos
2. Fundamentos: curvas elípticas
3. Protocolos, implementación y metodología
4. Resultados: los tres ejes
5. Conclusiones y trabajo futuro

Motivación y objetivos

Motivación

- RSA (años 70) sigue siendo el pilar de la clave pública, pero su seguridad exige claves **cada vez más grandes**.
- **1985**: Koblitz y Miller proponen, de forma independiente, usar **curvas elípticas** (ECC): misma seguridad con claves **mucho menores** $\Rightarrow$ idóneas para dispositivos limitados y alto rendimiento.
- **Horizonte cuántico**: el algoritmo de Shor rompe —en teoría demostrada— tanto RSA (factorización) como ECC (logaritmo discreto), pero exige un ordenador cuántico tolerante a fallos que **aún no existe**. La transición post-cuántica es **híbrida**: ECC no se retira.

Pregunta del trabajo

No basta con saber que ECC es “mejor”. ¿Cuánto vale esa ventaja en la práctica, y por qué?

Objetivo, aportación y los tres ejes

Objetivo: implementar ambos criptosistemas en C++17 y compararlos empíricamente con rigor.

Eje 1

RSA vs ECC

a igual nivel de seguridad
(NIST SP 800-57)

Eje 2

Afines vs Jacobianas

efecto de las coordenadas
proyectivas

Eje 3

$\mathbb{F}_p$ vs $\mathbb{F}_{2^m}$

cuerpos primos vs binarios

Aportación

Separar empíricamente lo que es **álgebra** (ventaja estructural, transferible) de lo que es **ingeniería** (cifras que dependen del entorno).

Fundamentos: curvas elípticas

¿Qué es una curva elíptica?

Sobre un cuerpo K (con $\text{char}(K) \neq 2, 3$), el conjunto de soluciones de la **ecuación de Weierstrass**

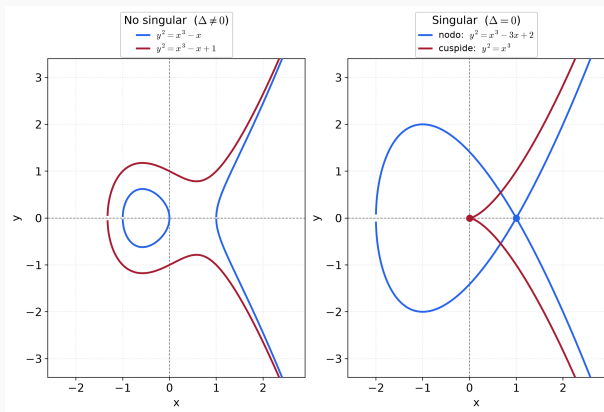
$$y^2 = x^3 + ax + b$$

junto con un **punto del infinito** $\mathcal{O}$.

No singularidad

$\Delta = -16(4a^3 + 27b^2) \neq 0$
(sin cúspides ni autointersecciones).

La forma corta exige $\text{char} \neq 2, 3$
(derivación en respaldo).



La ley de grupo: los puntos forman $(E(K), +)$

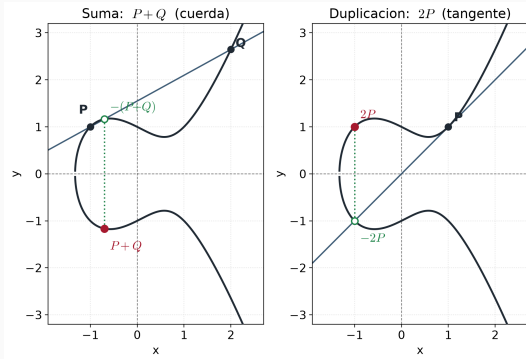
Grupo abeliano bajo la suma geométrica:

- **Suma** $P + Q$: la recta por P, Q corta en un tercer punto; su **reflexión** es $P + Q$.
- **Duplicación** $2P$: igual, con la **tangente** en P .
- **Neutro** $\mathcal{O}$; **inverso** $-P$ (reflexión).

Cumple los cinco axiomas (clausura, asociatividad, neutro, inverso, conmutatividad); la **asociatividad** es el no trivial (prueba en respaldo).

Teorema de Hasse

$$|\#E(\mathbb{F}_q) - (q + 1)| \leq 2\sqrt{q}$$



En cripto: subgrupo cíclico de **orden primo** n , con $\#E = h \cdot n$ (cofactor h pequeño). La dificultad del ECDLP es $\sim \sqrt{n}$.

Coordenadas: afines vs Jacobianas (motor del Eje 2)

Afines (x, y)

- Directas, pero **cada suma exige una inversión** en el cuerpo (para λ).
- La inversión es la operación **más cara**:
 $I \approx 80\text{--}100 M$.

Jacobianas $(X : Y : Z)$, $x = \frac{X}{Z^2}$, $y = \frac{Y}{Z^3}$

- No se normaliza $\Rightarrow$ se **difiere una única inversión al final**.
- Duplicación: $4M + 6S$ **sin I** (afín:
 $1I + 2M + 2S$).

Las fórmulas explícitas en $\mathbb{F}_p$ y $\mathbb{F}_{2^m}$ —ambas necesitan esa inversión para λ — están en respaldo.

Adelanto del hallazgo (Eje 2)

La teoría predice $\sim 8\times$; nosotros medimos $\sim 1.8\times$.
NTL/GMP hacen la inversión tan rápida que el cociente real $I/M \ll 80$, y el ahorro teórico se reduce.

El problema difícil (ECDLP) y su consecuencia

Multiplicación escalar kP con double-and-add, $O(\log k)$.

- **Fácil**: dados k, P , calcular $Q = kP$.
- **Inviabile (ECDLP)**: dados P, Q , hallar k .

Mejor ataque genérico (rho de Pollard): $O(\sqrt{n})$ en tiempo, $O(1)$ en memoria $\Rightarrow$ **exponencial en bits**. RSA en cambio factoriza en **subexponencial** $L_n[1/3]$.

k es la clave privada, $Q = kP$ la pública $\Rightarrow$ base de ECDH y ECDSA.

Seguridad	RSA	ECC	Razón
80 bits	1024 b	160 b	6×
112 bits	2048 b	224 b	9×
128 bits	3072 b	256 b	12×
192 bits	7680 b	384 b	20×
256 bits	15360 b	512 b	30×

NIST SP 800-57: la ventaja de ECC es **estructural** y **crece** con la seguridad.

Protocolos, implementación y metodología

Protocolos sobre el ECDLP: ECDH y ECDSA

ECDH (intercambio de claves)

- Alice: d_A , $Q_A = d_A G$; Bob: d_B , $Q_B = d_B G$.
- Secreto común $S = d_A Q_B = d_B Q_A = d_A d_B G$.
- El intruso ve Q_A, Q_B , pero recuperar d es el ECDLP.

ECDSA (firma digital)

- Firma (r, s) : $r = x_{kG} \bmod n$,
 $s = k^{-1}(e + dr) \bmod n$.
- Verifica: $R' = u_1 G + u_2 Q$; aceptar si
 $x_{R'} \bmod n = r$.

El nonce k es crítico

Si se reutiliza o es predecible, se despeja la clave privada d (casos PlayStation 3 y Bitcoin). Solución: nonce determinista (RFC 6979).

Desarrollo completo, parámetros del dominio y prueba de corrección: en respaldo.

Arquitectura modular

- RSA con descifrado acelerado por CRT
- ECC afín y Jacobiano sobre $\mathbb{F}_p$
- ECC sobre cuerpos binarios $\mathbb{F}_{2^m}$
- SHA-256 desde cero (FIPS 180-4)
- RNG validado, cinco modos CLI

Stack

C++17

NTL + GMP

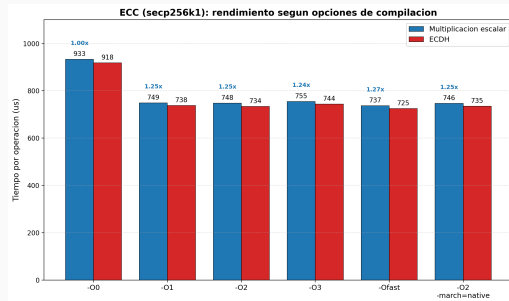
Docker (reproducibilidad)

SHA-256 a mano: entender el hash de ECDSA y evitar que librerías externas contaminen los tiempos.

Metodología: convertir medidas en ciencia

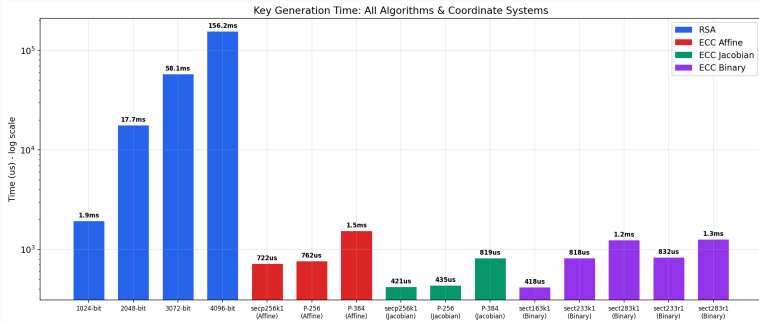
- **Docker**: mismo entorno, reproducible.
- **RNG con semilla fija**: comparación **pareada**.
- **Mediana** + calentamiento (3 descartados, 100 iteraciones).
- A **igual nivel de seguridad** (NIST), nunca a igual tamaño de clave.

Flags: de `-O0` a `-O1` se gana $\sim 1.25\times$; por encima de `-O2` las mejoras son marginales (el cálculo pesado vive en GMP, ya precompilado) y `-O3` mete ruido. Estándar del TFG: `-O2`.



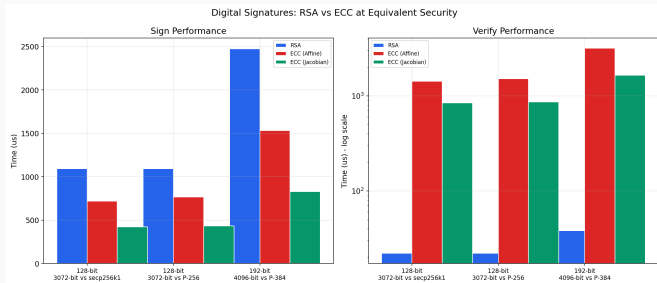
Resultados: los tres ejes

Eje 1 – ECC genera claves $\sim 138\times$ más rápido que RSA



RSA debe **buscar primos grandes** por ensayo y error; ECC solo **elige un escalar**. RSA-3072 ≈ 58 ms frente a ECC Jacobiano ≈ 0.42 ms: $\sim 138\times$ (escala logarítmica).

Eje 1 – firma y verificación: sin ganador absoluto

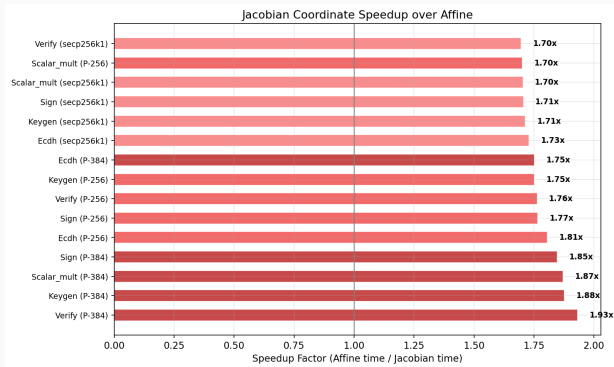


- **Firma:** ECC $\sim 2.6\times$ más rápida.
- **Verificación:** RSA $\sim 38\times$ más rápida.

RSA gana en verificación por su exponente público $e = 65537$ (solo 2 bits a 1).

No hay ganador absoluto: depende del perfil de uso.

Eje 2 – Jacobianas: $1.8\times$ medido frente a $8\times$ teórico

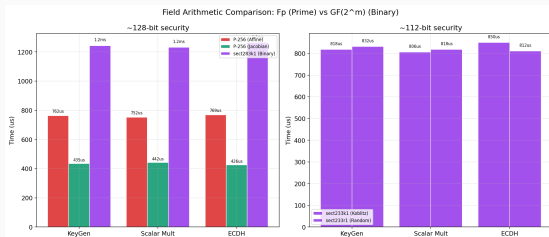


- Las Jacobianas **evitan la inversión modular** (visto en fundamentos).
- Medido: **$1.70\text{--}1.93\times$** (media $\sim 1.8\times$).

La brecha ES el hallazgo

La teoría predice $\sim 8\times$; NTL/GMP hacen la inversión barata, y el ahorro real es mucho menor.

Eje 3 y validación: artefacto de software, no de álgebra



Eje 3: a 128 bits el binario resulta $\sim 1.7\times$ **más lento** que el primo (afín). GF2E de NTL es genérico frente al ensamblador de GMP; además solo se midió en afín. Con hardware dedicado (PCLMULQDQ) el binario podría ganar.

Validación (OpenSSL): $14\text{--}34\times$ más rápido en P-256 (asm por curva + blinding); RSA-4096 verificación **a la par**. Que “ganemos” en RSA es un **artefacto de medida**, no una victoria.

Mensaje común

Estas cifras dependen del **entorno**, no del álgebra.

Conclusiones y trabajo futuro

Estructural (transferible)

La ventaja de ECC en tamaño de clave es **matemática** (ECDLP exponencial) y **crece** con la seguridad.

De implementación (del entorno)

Las cifras de velocidad (Jacobiano $1.8\times$, binario lento, “ganar” a OpenSSL) dependen de compilador, arquitectura y bibliotecas.

Limitaciones conscientes (de alcance, no olvidos): no es de **tiempo constante** (no apto para producción); RSA **de libro** (sin OAEP/PSS); el modelo Jacobiano solo se implementó en $\mathbb{F}_p$.

El valor del trabajo: separar el álgebra de la ingeniería.

ECC hoy y el horizonte post-cuántico

- **Web (TLS 1.3):** ~97–98 % de los handshakes usan ECDHE; **X25519** por defecto en navegadores.
- **Passkeys/FIDO2:** ECDSA P-256. **Signal, WhatsApp, SSH, WireGuard:** Curve25519 / Ed25519.
- **Criptomonedas:** Bitcoin y Ethereum → **secp256k1** (la curva de Koblitz del TFG).

La transición es híbrida, no una retirada

Shor rompe **RSA y ECC por igual** (NIST 2024: ML-KEM, ML-DSA, SLH-DSA). Pero X25519MLKEM768 ya cubre >40 % del tráfico HTTPS humano en Cloudflare: la parte clásica **sigue siendo ECC**. El banco de pruebas construido es **directamente reutilizable** para comparar los esquemas PQC frente a ECC.

Trabajo futuro: padding seguro (OAEP/PSS); optimizaciones (NAF, precomputación); portar el binario a coordenadas Jacobianas.

Gracias por su atención

`github.com/leonfullxr/ECC-Thesis`

Respaldo – de la ecuación general a la forma corta

Toda curva elíptica parte de la **ecuación general de Weierstrass**:

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6$$

Sobre cuerpos con $\text{char}(K) \neq 2, 3$ se simplifica en dos pasos:

1. **Completar el cuadrado** en y $[y \mapsto y - \frac{1}{2}(a_1x + a_3)]$, **requiere dividir por 2**:

$$y^2 = x^3 + a'_2x^2 + a'_4x + a'_6.$$

2. **Eliminar el término cuadrático** $[x \mapsto x - \frac{a'_2}{3}]$, **requiere dividir por 3**:

$y^2 = x^3 + ax + b$

.

Los divisores 2 y 3 son exactamente la razón de exigir $\text{char}(K) \neq 2, 3$.

Respaldo – curvas sobre cuerpos binarios $\mathbb{F}_{2^m}$

En char 2 **no se puede dividir entre 2**: falla el primer paso. Para curvas **ordinarias** (no supersingulares) se llega a la forma usada en el TFG:

$$\boxed{y^2 + xy = x^3 + ax^2 + b}, \quad b \neq 0$$

- Forma de las curvas **SEC 2**: sect163k1, sect233k1, sect283k1 (Koblitz) y sect233r1, sect283r1 (aleatorias).
- Koblitz: $a \in \{0, 1\}$, $b = 1$; admiten optimización por el endomorfismo de Frobenius (no aprovechada en nuestra versión afín).
- No singular $\iff b \neq 0$ (aquí $\Delta = b$).

Respaldo – fórmulas de suma en $\mathbb{F}_p$ y $\mathbb{F}_{2^m}$

Cuerpo primo $y^2 = x^3 + ax + b$

Suma ($P \neq Q$): $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$

$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$

Duplicación ($2P$): $\lambda = \frac{3x_1^2 + a}{2y_1}$

$x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$

$-P = (x_1, -y_1)$

Cuerpo binario $y^2 + xy = x^3 + ax^2 + b$

Suma ($P \neq Q$): $\lambda = \frac{y_1 + y_2}{x_1 + x_2}$

$x_3 = \lambda^2 + \lambda + x_1 + x_2 + a$

$y_3 = \lambda(x_1 + x_3) + x_3 + y_1$

Duplicación ($2P$): $\lambda = x_1 + \frac{y_1}{x_1}$

$x_3 = \lambda^2 + \lambda + a, \quad y_3 = x_1^2 + (\lambda + 1)x_3$

$-P = (x_1, x_1 + y_1)$; restar = sumar

Ambas requieren una inversión en el cuerpo para calcular λ .

Respaldo – atacar el ECDLP: rho de Pollard

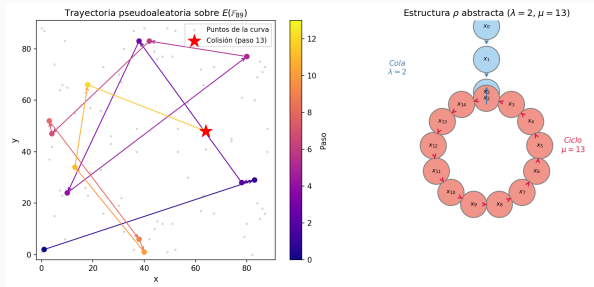
Mejor ataque **genérico** conocido:

- Paseo **pseudoaleatorio** sobre el grupo de puntos.
- Por la **paradoja del cumpleaños**, aparece pronto una **colisión**: el camino entra en un ciclo (forma de ρ).
- Detección de ciclo con poca memoria (Floyd). De la colisión se despeja k .

Coste

$O(\sqrt{n})$ tiempo, $O(1)$ memoria $\Rightarrow$

exponencial en bits.



Respaldo – parámetros del dominio

Séxtupla acordada por todas las partes: $T = (p, a, b, G, n, h)$.

Requisitos de seguridad

- p : primo que define $\mathbb{F}_p$.
- a, b : coeficientes de $y^2 = x^3 + ax + b$.
- G : punto generador; n : su orden.
- $h = \#E(\mathbb{F}_p)/n$: cofactor.
- n primo (evita Pohlig–Hellman).
- h pequeño, ideal 1.
- No anómala: $\#E(\mathbb{F}_p) \neq p$.
- Grado de inmersión alto (evita MOV).

En el TFG se usan curvas estándar **NIST/SECG**, ya auditadas.

Respaldo – ECDH en detalle



Ambos calculan el **mismo secreto** (asociatividad y conmutatividad):

$$S = d_A Q_B = d_B Q_A = d_A d_B G \implies \text{clave de sesión} = \text{KDF}(x_S)$$

En modo efímero (ECDHE) da **secreto perfecto hacia adelante**; no autentica por sí solo (hace falta firma/certificado contra MITM).

Firma (clave privada d)

1. $e = H(m)$
2. nonce $k \xleftarrow{\$} [1, n - 1]$
3. $R = kG = (x_R, y_R)$, $r = x_R \text{ mód } n$
4. $s = k^{-1}(e + d r) \text{ mód } n$

Firma: (r, s)

Verificación (clave pública $Q = dG$)

1. $e = H(m)$, $w = s^{-1} \text{ mód } n$
2. $u_1 = e w \text{ mód } n$, $u_2 = r w \text{ mód } n$
3. $R' = u_1 G + u_2 Q$
4. aceptar si $x_{R'} \text{ mód } n = r$

Respaldo – corrección de la verificación ECDSA

Si la firma es honesta, $s = k^{-1}(e + d r)$ mód n , luego

$$k = s^{-1}(e + d r) = \underbrace{s^{-1}e}_{u_1} + \underbrace{s^{-1}r}_{u_2} d = u_1 + u_2 d \quad (\text{mód } n).$$

Por tanto $R' = u_1 G + u_2 Q = u_1 G + u_2 d G = (u_1 + u_2 d) G = k G = R$, de donde $x_{R'} \text{ mód } n = x_R \text{ mód } n = r$ y la firma se acepta.



Respaldo – validación frente a OpenSSL

	OpenSSL vs propia
P-256 (operaciones)	14–34× más rápido
P-384	1.7–4× más rápido
RSA-4096 verificación	a la par (1.08×)

Por qué la diferencia

Ensamblador a mano + **blinding** contra canales laterales que mi versión académica no incorpora.

Objetivo del TFG: **entender y comparar**, no competir con código de producción.

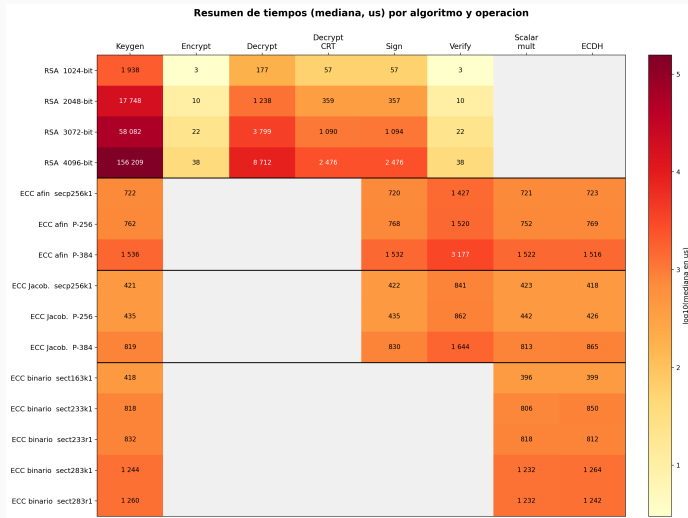
Respaldo – CRT en el descifrado RSA

Se calcula $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, $q_{inv} = q^{-1} \bmod p$. Recombinación de Garner:
 $m_1 = c^{d_p} \bmod p$, $m_2 = c^{d_q} \bmod q$, $h = q_{inv}(m_1 - m_2) \bmod p$, $m = m_2 + hq$. Acelera $\approx 3.5\times$ (dos exponenciaciones a la mitad de bits).

Respaldo – la asociatividad de la ley de grupo

$(E(K), +)$ es grupo abeliano. Neutro $\mathcal{O}$, inversos (reflexión) y conmutatividad son inmediatos. El axioma no trivial es la **asociatividad**: se prueba con el teorema de Cayley–Bacharach o, más elegantemente, identificando E con su grupo de clases de divisores $\text{Pic}^0(E)$ (Riemann–Roch).

Respaldo – panorama completo de tiempos



Respaldo – razón de rendimiento RSA/ECC

